UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 19

Proyectos de múltiples archivos

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 Objetivos de la sesión **2**

2 Prerrequisitos **2**

3 Arranque y preguntas de sondeo **3**

4 Declaración versus definición **3**
4.1 Explicación central 3
4.2 Guards de inclusión 3

5 Compilación por etapas **4**
5.1 Demo en vivo: los dos pasos 4
5.2 La línea que no para de crecer 5

6 Make: del Makefile mínimo a las variables **6**
6.1 Secuencia de escritura en vivo 6
6.2 Makefile completo de referencia 7

7 CMake **8**

8 Proyecto ejemplo: sistema de personas **10**

9 Errores frecuentes **10**

10 Soluciones a los ejercicios **11**

11 Conexiones con temas posteriores **13**

12 Rúbrica de ejercicios **14**

13 Materiales y recursos **14**

Objetivos de la sesión

Al finalizar la sesión el alumno será capaz de:

1. Explicar la diferencia entre declaración y definición en C++ y por qué los headers contienen solo declaraciones.
2. Escribir un archivo `.h` correcto con guard de inclusión.
3. Describir los dos pasos de compilación (`.cpp` → `.o` → ejecutable) y el rol del enlazador.
4. Escribir un `Makefile` básico partiendo de la línea de compilación directa y evolucionarlo con variables.
5. Crear un `CMakeLists.txt` mínimo y seguir el flujo *out-of-source build*.
6. Leer un proyecto de varios módulos e identificar el grafo de dependencias entre archivos.

90 minutos. Distribución sugerida:

10 min	gancho histórico + preguntas de arranque
15 min	headers: declaración vs. definición, guards
20 min	demo en vivo: compilar por etapas con `g++` + la línea que no para de crecer
25 min	Make: del Makefile mínimo a las variables
15 min	CMake: `CMakeLists.txt` y flujo de trabajo
5 min	proyecto de personas + asignación de ejercicios

Los alumnos siguen la demo en el cuaderno Jupyter (`sesiones/sesion19_proyectos_multiarc`
Los ejercicios se terminan en casa.

Prerrequisitos

- Compilar y ejecutar un programa de un solo archivo con `g++` (sesiones 1–2).
- Funciones: declaración y llamada (sesión 4).
- Apuntadores y arrays como parámetros (sesión 14).
- Familiaridad básica con terminal: `ls`, `cd`, `cat`, redirección.

Esta sesión no requiere recursión ni conocimiento de STL. Es intencionalmente accesible: el arco conceptual va de “un archivo” a “varios archivos”, y todo el herramental (headers, `.o`, Make, CMake) es motivado por ese problema.

Señal de alerta en la demo: si un alumno no recuerda cómo compilar con `g++`, detenerse brevemente en los flags más comunes (`-o`, `-Wall`, `-std=c++17`) antes de agregar `-c`.

Arranque y preguntas de sondeo

1. "¿Cuántos archivos tiene el proyecto más grande que han escrito?"
Respuesta esperada: uno. Si alguien dice más, pedirle que explique cómo lo organizó. Seguir con: "¿cómo saben en cuál archivo está la función que necesitan?"

2. "Si tienen un archivo de 2 000 líneas y cambian una función de 3 líneas, ¿qué tiene que hacer la computadora?"
Respuesta esperada: recompilar todo. Seguir con: "¿y si pudiéramos recompilar solo esa función?" — eso es lo que aprenderemos hoy.

3. "¿Cómo creen que funciona el kernel de Linux, que tiene más de 30 millones de líneas?"
Respuesta esperada: muchos archivos. Dato de gancho: el kernel tiene ≈70 000 archivos `.c` y su `Makefile` tiene miles de líneas. Sin herramientas de gestión de compilación sería imposible de mantener.

Declaración versus definición

- ## Explicación central

  La distinción es sencilla pero se confunde con frecuencia:

  - **Declaración:** le dice al compilador que algo *existe* y cuál es su forma (tipo de retorno, parámetros). No genera código ejecutable. Se puede repetir cuantas veces sea necesario.
  - **Definición:** proporciona el cuerpo. Solo puede aparecer **una vez** en todo el programa (**ODR**: *One Definition Rule*).

"Una *declaración* es como el índice de un libro: te dice que existe el capítulo 5 y en qué página está. La *definición* es el capítulo completo. Puedes imprimir el índice mil veces (en cada archivo que lo necesite), pero el capítulo completo solo existe una vez."

Esta analogía funciona bien para motivar el patrón `.h` = declaraciones, `.cpp` = definiciones.

- ## Guards de inclusión

Crear un `funciones.h` sin guard que declare `void saludar(char*);`. En `main.cpp`, incluirlo dos veces:

```
#include "funciones.h"
#include "funciones.h"   // segunda inclusion deliberada
```

Con solo declaraciones no ocurre ningún error (las declaraciones se pueden

repetir). Ahora poner una definición en el header:

```
void saludar(char* n) { cout << "Hola " << n; }
```

Compilar y mostrar el error:

```
error: redefinition of 'void saludar(char*)'
```

Agregar el guard y demostrar que desaparece. Luego quitar la definición del header (regla: solo declaraciones) y verificar que tampoco hay error sin guard.

Por qué vale la pena hacer esto: los alumnos recuerdan el guard porque lo ven solucionar un error concreto, no porque alguien les dijo que lo pusieran.

¿Cuál es mejor?
`#pragma once` es más simple y elimina el riesgo de escribir mal el nombre de la macro (error clásico: `FUNCIONES_H` en el `#ifndef` y `FUNCIONES_HH` en el `#define`). El guard manual es el estándar ISO histórico (aparece en todos los tutoriales y código existente). Para este curso, ambas formas son aceptables.

¿Puede un `.h` contener definiciones?
Sí, en dos casos: funciones `inline` y plantillas (*templates*). Ambos tienen reglas especiales en la ODR que permiten múltiples definiciones idénticas. Para este curso, la regla simple es: solo declaraciones en el `.h`.

Compilación por etapas

- **Demo en vivo: los dos pasos**

Trabaja con los archivos de `materialInicial/sesion19/` (`main.cpp`, `funciones.h`, `funciones.cpp`).

1. Mostrar los tres archivos con `cat` o en el editor.
2. Compilar solo `funciones.cpp`:

   ```
   g++ -c funciones.cpp -o funciones.o
   ls -lh funciones.o
   ```

3. Intentar ejecutar el `.o` directamente:

   ```
   ./funciones.o
   ```

   Da error — no es un ejecutable. Preguntar: "¿qué falta?" El punto de entrada (`main`). Eso es exactamente lo que aporta el enlazador.
4. Opcional: `nm -demangle funciones.o` para mostrar los símbolos definidos (`T`) en el objeto.

5. Compilar `main.cpp`:

```
g++ -c main.cpp -o main.o
nm --demangle main.o | grep saludar
```

   Señalar que `saludar` aparece como `U` (*undefined*) en `main.o`: está declarada pero no definida ahí.

6. Enlazar:

```
g++ main.o funciones.o -o programa
./programa
```

7. Modificar `funciones.cpp` (cambiar el mensaje). Mostrar que solo hay que recompilar ese archivo:

```
g++ -c funciones.cpp -o funciones.o
g++ main.o funciones.o -o programa
./programa
```

   `main.o` no se volvió a compilar. Preguntar: "¿por qué no necesitamos recompilar `main.o`?"

- **La línea que no para de crecer**

Después de la demo, mostrar cómo se ve la línea de compilación para el proyecto de personas (6 módulos):

```
g++ -std=c++17 -Wall -Wextra -o programa \
    main.cpp auxiliar.cpp obtener_datos.cpp \
    mate.cpp buscar.cpp
```

Preguntar: "¿Qué pasa cuando el proyecto tenga 20 archivos? ¿O cuando necesitemos agregar `-O2` para optimización? ¿O linkear una librería externa?" Mostrar la versión inflada:

```
g++ -std=c++17 -Wall -Wextra -O2 -I./include \
    -I./lib/json/include \
    main.cpp auxiliar.cpp obtener_datos.cpp    \
    mate.cpp buscar.cpp graficas.cpp reportes.cpp \
    -L./lib -ljson -lm -o programa
```

Pausa. "Hay una solución mejor. Se llama `Makefile`."

Por qué funciona esta transición: el alumno ya vivió la incomodidad de la línea larga durante la demo. La motivación es visceral, no abstracta.

¿Qué es un "error de enlace" (*linker error*)?
Ocurre cuando el linker no encuentra la definición de algo declarado. El mensaje típico es:

```
undefined reference to 'saludar(char*)'
```

Causa más común: olvidar incluir un `.cpp` en el comando de enlace.

¿Por qué el `.h` no se compila directamente?
El `.h` no contiene definiciones; el preprocesador lo copia textualmente en cada `.cpp` que lo incluye. Técnicamente puedes hacer `g++ -c funciones.h`, pero genera un objeto vacío o inútil.

¿Qué significa el flag `-I./include`?
Le dice al compilador dónde buscar headers adicionales. Sin ese flag, `#include "json.hpp"` fallaría si el archivo no está en el directorio actual.

Make: del Makefile mínimo a las variables

- **Secuencia de escritura en vivo**

Escribe el Makefile en vivo, paso a paso. No muestres el resultado final de entrada — la evolución gradual es el punto pedagógico central de esta sesión.

Paso 1 — Solo la línea de compilación (5 min)
Crear `Makefile` con el editor y escribir:

```
programa:
 g++ -std=c++17 -Wall -Wextra -o programa \
     main.cpp auxiliar.cpp obtener_datos.cpp \
     mate.cpp buscar.cpp
```

Ejecutar `make`. Preguntar: "¿Qué ganamos respecto al comando directo?" Respuesta: no tenemos que *recordar* ni *teclear* la línea — solo `make`.

Señalar el Tab antes del comando. Borrar el Tab, reemplazar con espacios y ejecutar de nuevo para provocar el error:

```
Makefile:2: *** missing separator.  Stop.
```

Restaurar el Tab.

Paso 2 — Variable para el compilador (3 min)
Agregar `CXX = g++` arriba y sustituir en el comando. Preguntar: "¿Por qué es útil poner el compilador en una variable?" Si alguien trabaja con `clang++` o necesita cross-compilar, solo cambia una línea.

Paso 3 — Variable para los flags (3 min)
Agregar `CXXFLAGS = -std=c++17 -Wall -Wextra`. Enfatizar: cuando el proyecto crezca y se necesite agregar `-O2` o `-g`, se edita solo `CXXFLAGS`, no cada regla.

Paso 4 — Variables para archivos y target (4 min)
Agregar `TARGET` y `SRCS`:

```
CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra
TARGET   = programa
SRCS     = main.cpp auxiliar.cpp obtener_datos.cpp \
           mate.cpp buscar.cpp

$(TARGET): $(SRCS)
 $(CXX) $(CXXFLAGS) $(SRCS) -o $(TARGET)
```

Preguntar: "Ahora quiero agregar `graficas.cpp`. ¿Qué línea toco?" Solo `SRCS`.

Paso 5 — Introducir OBJS y recompilación selectiva (10 min)
Ejecutar `make` dos veces seguidas para mostrar que la segunda vez recompila todo aunque nada cambió. Preguntar: "¿Cómo podría Make saber que ya está compilado?" Llevar hacia la idea de fechas y archivos intermedios.
Evolucionar a la versión con `OBJS`:

```
OBJS = main.o auxiliar.o obtener_datos.o mate.o buscar.o

$(TARGET): $(OBJS)
 $(CXX) $(OBJS) -o $(TARGET)

%.o: %.cpp
 $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
 rm -f $(OBJS) $(TARGET)
```

Ejecutar `make`. Luego:

```
touch auxiliar.cpp    # simula una edición en auxiliar.cpp
make
```

Mostrar que solo `auxiliar.o` se recompila. Luego:

```
touch persona.h       # simula un cambio en el header compartido
make
```

Mostrar que Make *no* detecta esto sin dependencias explícitas de headers — solo reenlaza el ejecutable. Decir: "¿Es correcto? No. Pero Fix completo con dependencias de headers es para el ejercicio." (El alumno lo resuelve en el ejercicio 2.)

Terminar con `make clean` y verificar que desaparecen los artefactos.

• Makefile completo de referencia

```
CXX      = g++
```

```
CXXFLAGS = -std=c++17 -Wall -Wextra
TARGET   = programa
SRCS     = main.cpp auxiliar.cpp obtener_datos.cpp mate.cpp buscar.cpp
OBJS     = $(SRCS:.cpp=.o)

$(TARGET): $(OBJS)
 $(CXX) $(OBJS) -o $(TARGET)

%.o: %.cpp
 $(CXX) $(CXXFLAGS) -c $< -o $@

clean:
 rm -f $(OBJS) $(TARGET)
```

Error "missing separator":
El comando empieza con espacios, no con Tab. En VS Code se ve igual visualmente; decir a los alumnos que abran el archivo en `cat -A` para ver los caracteres ocultos (`^I` = Tab, espacios = espacios).

¿Por qué no usar simplemente `g++ *.cpp` siempre?
Funciona para compilar, pero recompila *todo* en cada invocación. En proyectos de 100+ archivos con 5+ minutos de compilación, es inaceptable.

¿Qué significa `$(SRCS:.cpp=.o)`?
Es una sustitución de texto en Make: reemplaza la extensión `.cpp` por `.o` en cada elemento de `SRCS`. Equivale a escribir `OBJS = main.o auxiliar.o ...` a mano.

¿Cómo agrego dependencias de headers?
Declarando el `.o` como objetivo con sus `.h` como dependencias:

```
auxiliar.o: auxiliar.cpp auxiliar.h persona.h
```

Con esto, si `persona.h` cambia, Make recompila `auxiliar.o`. Sin esta línea, Make no lo sabe. (Esto es el ejercicio 2.)

¿Hace Make algo con `clean`?
`clean` no es una regla especial de Make — es solo un objetivo sin archivo correspondiente. Make lo ejecuta cuando escribes `make clean`. La convención del nombre es universal.

CMake

1. Crear `CMakeLists.txt` en el directorio del proyecto:

```
cmake_minimum_required(VERSION 3.10)
project(MiProyecto VERSION 1.0)
set(CMAKE_CXX_STANDARD 17)
```

```
set(CMAKE_CXX_STANDARD_REQUIRED True)
add_executable(programa
    main.cpp
    auxiliar.cpp
    obtener_datos.cpp
    mate.cpp
    buscar.cpp
)
```

2. Crear el directorio de build y configurar:

   ```
   mkdir build && cd build
   cmake ..
   ```

   Señalar los archivos generados: `Makefile`, `CMakeCache.txt`, `CMakeFiles/`.

3. Compilar y ejecutar:

   ```
   make
   ./programa
   ```

4. Regresar al directorio fuente (`cd ..`) y mostrar que está limpio: solo `CMakeLists.txt`, los `.cpp` y los `.h`. Ningún `.o` ni `Makefile` generado ensucia el directorio de código.

5. Preguntar: "¿qué poner en `.gitignore`?" Respuesta: `build/`.

¿Para qué sirve `cmake_minimum_required`?
Protege contra versiones antiguas de CMake que no soporten las funciones usadas en el archivo. Buena práctica: poner la versión mínima real que necesita el proyecto.

¿Ninja vs. Make en CMake?
`cmake -GNinja ..` genera archivos Ninja en lugar de Makefile. Ninja es más rápido en proyectos grandes (mejor paralelismo). Para este curso, Make es suficiente.

¿Cómo agrego flags de compilación en CMake?

```
target_compile_options(programa PRIVATE -Wall -Wextra)
```

O globalmente con `add_compile_options(-Wall)`.

"Prefiero no usar `mkdir build / cd build`, ¿hay otra forma?"
Sí: `cmake -B build` desde el directorio raíz genera el build en `build/` sin necesidad de `cd`. Luego `cmake -build build` compila sin entrar al directorio. Más cómodo en proyectos donde se abre el editor en la raíz.

¿Cuándo usar Make y cuándo CMake?
Make: proyecto propio que solo se usará en Linux/Mac, equipo pequeño, simplicidad máxima. CMake: proyecto que debe funcionar en Windows, que se integrará con un IDE (CLion, VS, etc.), o que crecerá a más de un par de directorios. La industria usa CMake casi universalmente en proyectos C++ modernos.

Proyecto ejemplo: sistema de personas

El código en `materialInicial/sesion19/proyecto1/` tiene inconsistencias que conviene conocer antes de la sesión. El objetivo es usar el proyecto para *leer* el grafo de dependencias, no para compilarlo.

- **Firmas inconsistentes en `buscar.h` vs. `buscar.cpp`:** el header declara tres parámetros en algunas funciones pero la implementación usa dos. Código refactorizado a medias.
- **Funciones duplicadas:** `buscar.cpp` contiene implementaciones de `promedio_edades` y `promedio_salarios` que ya están en `mate.cpp`. Si se enlazan juntos, falla con error de ODR.
- **Include incorrecto en `main.cpp`:** incluye `"obtener.h"` pero el archivo se llama `obtener_datos.h`.

Sugerencia: mostrar el grafo de inclusiones en el pizarrón y señalar por qué `persona.h` está en el centro. Si se quiere compilar en clase, corregir los errores antes.

Errores frecuentes

1. Poner definiciones en el header.
Síntoma: el proyecto compila bien si solo `main.cpp` incluye el `.h`, pero falla al enlazar con un segundo `.cpp`. Error: `multiple definition of 'f()'`.
Remedio: mover la definición al `.cpp`; en el `.h` dejar solo la declaración.

2. Olvidar el Tab en el Makefile.
Error: `Makefile:N: *** missing separator. Stop.`
Remedio: el comando de la regla *siempre* empieza con Tab. En VS Code, verificar que el archivo no esté en modo "Indent with spaces".

3. No incluir el `.h` propio en el `.cpp`.
Síntoma: el `.cpp` compila solo, pero al enlazar con `main.cpp` (que sí incluye el `.h`) el linker reporta que las firmas no coinciden. Remedio: el primer `#include` en cualquier `.cpp` debe ser su propio `.h`. Esto hace que el compilador verifique que la declaración y la definición son consistentes.

4. Usar <> para headers propios.

Síntoma: `fatal error: funciones.h: No such file or directory.`
Remedio: headers del proyecto van con `""`, no con <>.

5. Olvidar agregar el `.cpp` al Makefile o CMakeLists.
Síntoma: el código compila pero hay errores de enlace para funciones del módulo nuevo.
Remedio: todo `.cpp` que define funciones usadas por otros debe aparecer en la lista de fuentes del sistema de compilación.

Soluciones a los ejercicios

El alumno debe mostrar cuatro versiones sucesivas del Makefile.

Versión 1 — Solo la línea de compilación:

```
programa:
 g++ -std=c++17 -Wall -Wextra -o programa \
     main.cpp funciones.cpp
```

Versión 2 — Agrega `CXX`:

```
CXX = g++

programa:
 $(CXX) -std=c++17 -Wall -Wextra -o programa \
     main.cpp funciones.cpp
```

Versión 3 — Agrega `CXXFLAGS` y `TARGET`:

```
CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra
TARGET   = programa

$(TARGET):
 $(CXX) $(CXXFLAGS) -o $(TARGET) main.cpp funciones.cpp
```

Versión 4 — Agrega `SRCS`:

```
CXX      = g++
CXXFLAGS = -std=c++17 -Wall -Wextra
TARGET   = programa
SRCS     = main.cpp funciones.cpp

$(TARGET): $(SRCS)
 $(CXX) $(CXXFLAGS) $(SRCS) -o $(TARGET)
```

Criterio de evaluación: cada versión debe ser funcional (compilar el proyecto) y mostrar un avance claro respecto a la anterior.

Modificar `funciones.cpp`:
Make recompila `funciones.o` y reenlaza el ejecutable. `main.o` no se toca porque su fecha es más reciente que la de `main.cpp`.

Modificar `funciones.h`:
Con el Makefile de la sección 5 (que tiene la regla patrón pero *no* declara dependencias de headers explícitas), Make **no** detecta el cambio en `funciones.h` porque ningún `.o` lo lista como dependencia. Solo recompila si el `.o` correspondiente es más nuevo que el `.h`, lo cual no siempre ocurre.

Para que funcione correctamente, hay que agregar:

```
main.o:      main.cpp funciones.h
funciones.o: funciones.cpp funciones.h
```

Ahora si `funciones.h` cambia, Make recompila **ambos** objetos.

Mismo archivo:

```
void f() {}
void f() {}   // error: redefinition of 'void f()'
```

Error de *compilación*. El compilador lo detecta antes de enlazar.

Archivos distintos: si `a.cpp` y `b.cpp` definen ambos `void f() {}`, cada archivo compila sin error, pero al enlazar:

```
error: multiple definition of 'f()'
```

Error de *enlace*. El linker detecta la violación de la ODR. Este es el tipo de error que tendría el `proyecto1` al enlazar `mate.o` y `buscar.o` (duplican la definición de `promedio_edades`).

estadisticas.h:

```
#ifndef ESTADISTICAS_H
#define ESTADISTICAS_H
#include "persona.h"

double salario_maximo(Persona arr[], int n);

#endif
```

estadisticas.cpp:

```
#include "estadisticas.h"

double salario_maximo(Persona arr[], int n) {
```

```
    double max = arr[0].salario;
    for (int i = 1; i < n; i++)
        if (arr[i].salario > max) max = arr[i].salario;
    return max;
}
```

Makefile — agregar a `SRCS`:

```
SRCS = main.cpp auxiliar.cpp obtener_datos.cpp \
       mate.cpp buscar.cpp estadisticas.cpp
```

CMakeLists.txt — agregar a `add_executable`:

```
add_executable(programa
    ...
    estadisticas.cpp
)
```

`make`: construye los objetivos cuyas dependencias son más recientes que ellos mismos, o los que no existen aún.

`make clean`: ejecuta `rm -f $(OBJS) $(TARGET)`. Elimina todos los artefactos. Útil cuando:

- Se cambia un flag de compilación (`CXXFLAGS`). Make no detecta cambios en variables, solo en fechas de archivos.
- Se quiere verificar que el proyecto compila desde cero ("compilación limpia").
- Los archivos `.o` son de una plataforma diferente (por ejemplo, se copió el proyecto de otro sistema).

Conexiones con temas posteriores

- **Programación orientada a objetos** — Las clases en C++ viven naturalmente en módulos: `Persona.h` / `Persona.cpp`. El patrón que ven hoy (declaración en `.h`, definición en `.cpp`) es exactamente el que usarán para cada clase del curso OOP.
- **Librerías estáticas y dinámicas** — Los archivos `.o` que se producen hoy son los bloques que se empaquetan en una librería (`.a` estática o `.so`/`.dll` dinámica).
- **Header-only libraries** — Muchas librerías modernas de C++ (Eigen, nlohmann/j-son, stb) son solo un `.h`. Explotan la excepción `inline`/templates de la ODR para no requerir un `.cpp`. El alumno ya tiene los conceptos para entender por qué eso es posible.
- **Compilación incremental en CI/CD** — Los sistemas de integración continua (GitHub Actions, GitLab CI) usan Make o CMake para recompilar solo lo cambiado en cada PR. El tiempo de pipeline se reduce por el mismo principio de fechas que vieron hoy.

Rúbrica de ejercicios

Criterio	Puntos	Evidencia
Ejercicio 1 — Evolución del Makefile		
Versión 1 funcional (solo línea)	5	`make` produce ejecutable
Versiones 2–4 muestran evolución clara	5	variables correctas, uso de `$(VAR)`
Ejercicio 2 — Observar recompilación		
Identifica correctamente qué recompila	5	con y sin dependencia de header
Explica por qué `.h` no se detecta	5	dependencias explícitas son necesarias
Ejercicio 3 — Doble definición		
Distingue error de compilación vs. enlace	5	mensaje de error correcto en cada caso
Explica con la ODR	5	referencia a la regla
Ejercicio 4 — Módulo estadisticas		
Guard correcto en el header	4	nombre consistente
`.cpp` incluye su propio `.h`	3	primera línea del `.cpp`
Agregado a Makefile y CMakeLists.txt	3	en ambos lugares
Ejercicio 5 — make vs. make clean		
Diferencia correcta	5	qué hace cada uno
Menciona el caso de cambio de flag	5	Make no detecta cambios en variables
Total	**50**	

Materiales y recursos

- **Cuaderno:** `sesiones/sesion19_proyectos_multiarchivo.ipynb`
- **Notas alumno:** `notas_alumno/pdf/sesion19_proyectos_multiarchivo.pdf`
- **Material inicial:** `materialInicial/sesion19/` (proyecto mínimo y proyecto1)